

Lab 1: Display Density Metrics & Image Array Mechanics in NumPy

Department of Computer Science — HCI & CG

Task 1: Display Pixel Density (PPI/DPI) Calculator

Objective

Write a Python script that calculates screen parameters and classifies display density based on physical screen attributes.

Python Code

```
import math

def calculate_display_metrics(w_px, h_px, d_inches):
    # Total pixel count
    total_pixels = w_px * h_px

    # Simplified aspect ratio using GCD
    gcd = math.gcd(w_px, h_px)
    aspect_w = w_px // gcd
    aspect_h = h_px // gcd

    # Diagonal pixel count
    p_d = math.sqrt(w_px**2 + h_px**2)

    # DPI / PPI calculation
    dpi = p_d / d_inches

    # Classify display density
    if dpi < 100:
        category = "Low Density (Standard Monitor)"
    elif dpi ≤ 200:
        category = "Medium Density (HD Display)"
    else:
        category = "High Density (Retina / Mobile)"

    return total_pixels, (aspect_w, aspect_h), dpi, category

# ===== Test Case 1: Desktop Monitor =====
w1, h1, d1 = 1920, 1080, 24
total1, aspect1, dpi1, cat1 = calculate_display_metrics(w1, h1, d1)

print("--- DISPLAY METRICS ANALYSIS (Desktop) ---")
print(f"Total Pixel Count : {total1:,} pixels")
print(f"Aspect Ratio      : {aspect1[0]}:{aspect1[1]}")
print(f"Calculated DPI     : {dpi1:.2f} DPI")
print(f"Density Category   : {cat1}")

# ===== Test Case 2: Smartphone =====
w2, h2, d2 = 1170, 2532, 6.1
total2, aspect2, dpi2, cat2 = calculate_display_metrics(w2, h2, d2)
```

```
print("
--- DISPLAY METRICS ANALYSIS (Smartphone) ---")
print(f"Total Pixel Count : {total2:,} pixels")
print(f"Aspect Ratio      : {aspect2[0]}:{aspect2[1]}")
print(f"Calculated DPI    : {dpi2:.2f} DPI")
print(f"Density Category  : {cat2}")
```

Output Screenshot

Desktop Monitor (1920 x 1080, 24")

```
--- DISPLAY METRICS ANALYSIS (Desktop) ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
```

Smartphone (1170 x 2532, 6.1")

```
--- DISPLAY METRICS ANALYSIS (Smartphone) ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
```

Task 2: Environment Setup & Synthetic Image Matrix Creation

Objective

Install Python graphics packages and construct a 3D image matrix from scratch.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Create a 300x400x3 image array filled with zeros
img = np.zeros((300, 400, 3), dtype=np.uint8)

# Step 2: Fill the four quadrants using array slicing
# Top-Left Quadrant: Pure Red [255, 0, 0]
img[0:150, 0:200] = [255, 0, 0]

# Top-Right Quadrant: Pure Green [0, 255, 0]
img[0:150, 200:400] = [0, 255, 0]

# Bottom-Left Quadrant: Pure Blue [0, 0, 255]
img[150:300, 0:200] = [0, 0, 255]

# Bottom-Right Quadrant: White [255, 255, 255]
img[150:300, 200:400] = [255, 255, 255]

# Step 3: Display and save the synthetic image
plt.figure(figsize=(6, 4.5))
plt.imshow(img)
plt.title("Synthetic Quadrant Image (300x400x3)")
plt.axis('off')
plt.tight_layout()
plt.savefig("task2_synthetic_image.png", dpi=150)
plt.show()

# Step 4: Print matrix metrics
print("--- SYNTHETIC MATRIX METRICS ---")
print(f"Array Shape (H, W, C) : {img.shape}")
print(f>Data Type           : {img.dtype}")
print(f"Total Elements      : {img.size:,} values")
print(f"Memory Footprint      : {img.nbytes:,} bytes ({img.nbytes / 1024:.2f} KB)")
```

Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type           : uint8
Total Elements      : 360,000 values
Memory Footprint      : 360,000 bytes (351.56 KB)
```

Task 2: Synthetic Quadrant Image (300x400x3)

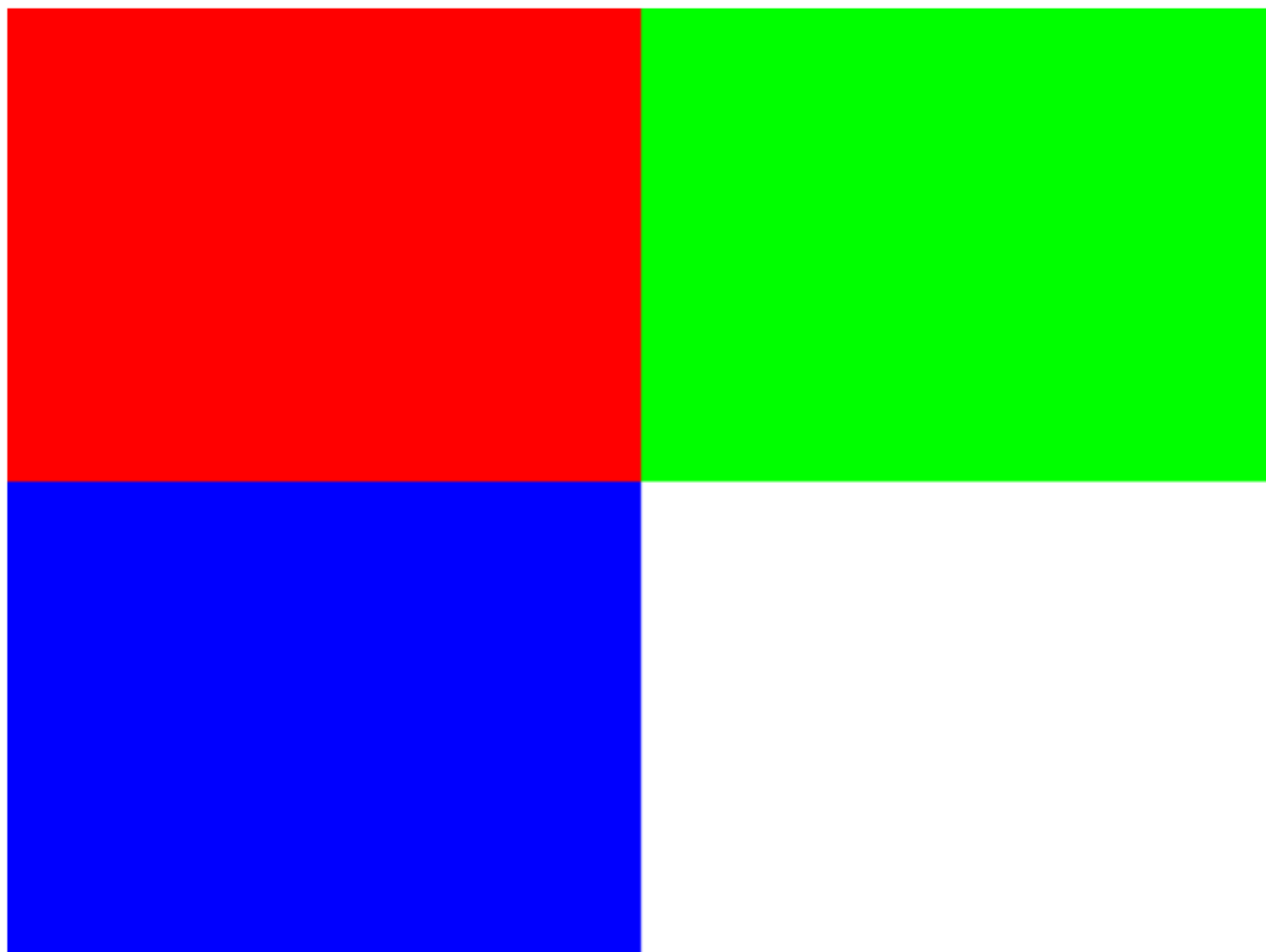


Figure 1: Synthetic Quadrant Image (Red, Green, Blue, White)

Task 3: Channel Slicing & Isolation

Objective

Load an image, extract individual 2D channel intensity grids, and isolate single-channel color views.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Step 1: Load the input image
img = np.array(Image.open("sample.jpg"))

# Step 2: Extract 2D intensity grids for each channel (Axis 2 slicing)
red_2d = img[:, :, 0] # Red channel
green_2d = img[:, :, 1] # Green channel
blue_2d = img[:, :, 2] # Blue channel

# Step 3: Calculate mean intensities
red_mean = red_2d.mean()
green_mean = green_2d.mean()
blue_mean = blue_2d.mean()

# Step 4: Construct 3D color arrays with only one channel active
red_only = np.zeros_like(img)
red_only[:, :, 0] = red_2d

green_only = np.zeros_like(img)
green_only[:, :, 1] = green_2d

blue_only = np.zeros_like(img)
blue_only[:, :, 2] = blue_2d

# Step 5: Display in a 2x3 subplot layout
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

# Top Row: Color images
axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only Channel")
axes[0, 0].axis('off')

axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only Channel")
axes[0, 1].axis('off')

axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only Channel")
axes[0, 2].axis('off')

# Bottom Row: Grayscale intensity maps
axes[1, 0].imshow(red_2d, cmap='gray')
axes[1, 0].set_title("Red Intensity (Grayscale)")
axes[1, 0].axis('off')
```

```

axes[1, 1].imshow(green_2d, cmap='gray')
axes[1, 1].set_title("Green Intensity (Grayscale)")
axes[1, 1].axis('off')

axes[1, 2].imshow(blue_2d, cmap='gray')
axes[1, 2].set_title("Blue Intensity (Grayscale)")
axes[1, 2].axis('off')

plt.suptitle("Channel Slicing & Isolation (2x3 Subplot Grid)", fontweight='bold')
plt.tight_layout()
plt.savefig("task3_channel_extraction.png", dpi=150)
plt.show()

# Step 6: Print summary
print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape      : {img.shape}")
print(f"Red Channel 2D Shape       : {red_2d.shape} | Mean Intensity: {red_mean:.2f}")
print(f"Green Channel 2D Shape     : {green_2d.shape} | Mean Intensity: {green_mean:.2f}")
print(f"Blue Channel 2D Shape      : {blue_2d.shape} | Mean Intensity: {blue_mean:.2f}")
print("Display Window           : Matplotlib 2x3 Subplot Grid Rendered.")

```

Output

```

--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape      : (300, 400, 3)
Red Channel 2D Shape     : (300, 400) | Mean Intensity: 126.69
Green Channel 2D Shape   : (300, 400) | Mean Intensity: 126.60
Blue Channel 2D Shape    : (300, 400) | Mean Intensity: 127.32
Display Window           : Matplotlib 2x3 Subplot Grid Rendered.

```

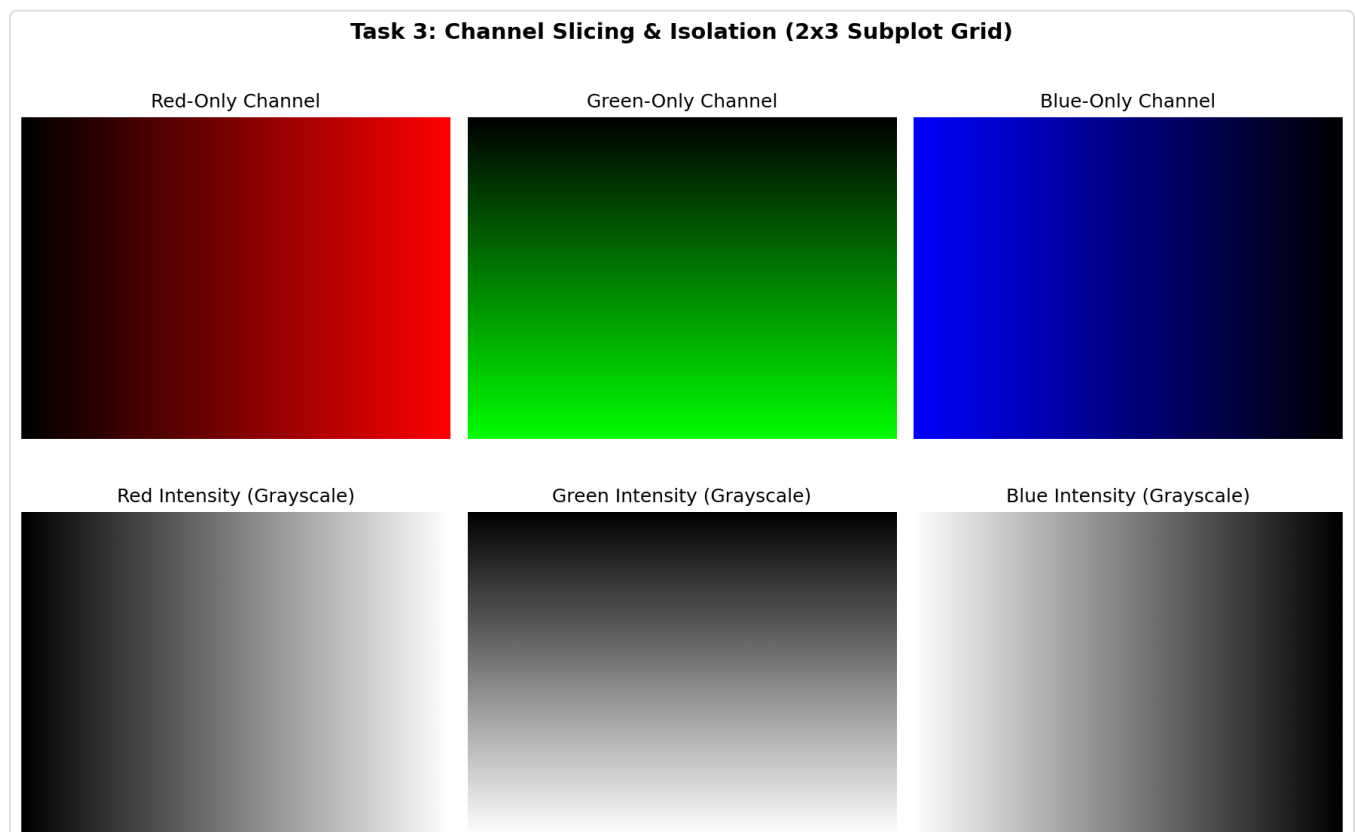


Figure 2: Channel Slicing & Isolation (2x3 Subplot Grid)

Task 4: Spatial Downsampling & Pixelation via Striding

Objective

Perform spatial downsampling directly in NumPy using slice steps.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Load the image (using the same sample image)
img = np.array(Image.open("sample.jpg"))
N = 8 # Stride factor

# Original metrics
orig_shape = img.shape
orig_memory = img.nbytes

# Step 1: Downsample using striding (take every N-th pixel)
downsampled = img[::N, ::N, :]
down_shape = downsampled.shape
down_memory = downsampled.nbytes

# Step 2: Re-expand using np.repeat to observe pixelation
reexpanded = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)
# Trim to original shape if needed
reexpanded = reexpanded[:orig_shape[0], :orig_shape[1], :]

# Step 3: Calculate percentage reductions
dim_reduction = ((orig_shape[0] - down_shape[0]) / orig_shape[0]) * 100
memory_reduction = ((orig_memory - down_memory) / orig_memory) * 100

# Step 4: Display comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(img)
axes[0].set_title(f"Original\n{orig_shape}")
axes[0].axis('off')

axes[1].imshow(downsampled)
axes[1].set_title(f"Downsampled (N={N})\n{down_shape}")
axes[1].axis('off')

axes[2].imshow(reexpanded)
axes[2].set_title(f"Re-expanded (Pixelated)\n{reexpanded.shape}")
axes[2].axis('off')

plt.suptitle("Spatial Downsampling & Pixelation via Striding", fontweight='bold')
plt.tight_layout()
plt.savefig("task4_downsampling.png", dpi=150)
plt.show()

# Step 5: Print analysis
print("--- DOWNSAMPLING ANALYSIS (N = 8) ---")
print(f"Original Shape      : {orig_shape} | Memory: {orig_memory:,} bytes")
```

```
print(f"Downsampled Shape : {down_shape} | Memory: {down_memory:,} bytes")
print(f"Re-expanded Shape : {reexpanded.shape} | Visual: Blocky Pixelation")
print(f"Dimension Reduction : {dim_reduction:.2f}% reduction per axis")
print(f"Memory Savings : {memory_reduction:.2f}% data reduction")
```

Output

```
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape      : (300, 400, 3) | Memory: 360,000 bytes
Downsampled Shape   : (38, 50, 3)   | Memory: 5,700 bytes
Re-expanded Shape    : (300, 400, 3) | Visual: Blocky Pixelation
Dimension Reduction  : 87.33% reduction per axis
Memory Savings       : 98.42% data reduction
```

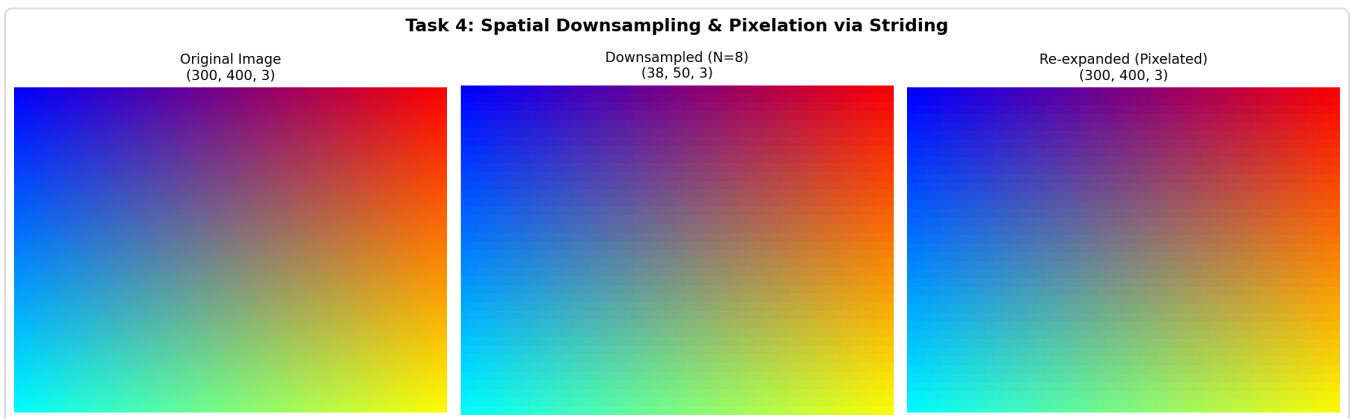


Figure 3: Original, Downsampled, and Re-expanded Pixelated Image

Short Answer

Question: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

Answer:

RGB (Red, Green, Blue) uses 3 channels, while RGBA adds an Alpha (transparency) channel, making it 4 channels total.

Memory Calculation:

- RGB: $H \times W \times 3$ bytes (e.g., $1920 \times 1080 \times 3 = 6,220,800$ bytes)
- RGBA: $H \times W \times 4$ bytes (e.g., $1920 \times 1080 \times 4 = 8,294,400$ bytes)

Difference: $4 - 3 = 1$ extra channel

Percentage increase: $(1 / 3) \times 100 = 33.33\%$

Therefore, adding an Alpha channel increases memory consumption by approximately 33% because each pixel now stores 4 values instead of 3, increasing the total number of elements by one-third.

Lab Deliverables Checklist

- ☐ Python source code (.py or .ipynb) for Tasks 1-4
- ☐ Screenshots of Task 1 output for desktop monitor (1920x1080, 24") and smartphone (1170x2532, 6.1")
- ☐ Matplotlib figure outputs for Task 3 and Task 4
- ☐ Short Answer: Why does adding an Alpha channel (RGBA) increase memory by 33% compared to RGB?